Practical No: 6

A) Aim: To implement a basic TCP socket communication model in Python that demonstrates how a server waits for client requests and how a client sends and receives data over a network connection.

Input: Server.py

```
import socket
HOST='127.0.0.1' #localhost
PORT=5000 #PORT NUMBER

server=socket.socket(socket.AF_INET,socket.SOCK_STREAM)

server.bind((HOST,PORT))

server.listen()

print(f'Server running on{HOST}:{PORT}...')

conn,addr=server.accept()
print('connected by',addr)

data=conn.recv(1024).decode()
print('client says:',data)

conn.sendall('hello from server!'.encode())

conn.close()
server.close()
```

Output:

```
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>cd AppData\Local\Programs\Python\Python313

C:\Users\User\AppData\Local\Programs\Python\Python313>python server.py
Server running on127.0.0.1:5000...
```

Input: client.py

```python
import socket
HOST='127.0.0.1' #server ip
PORT=5000 #server port

client=socket.socket(socket.AF_INET,socket.SOCK_STREAM)

client.connect((HOST,PORT))

client.sendall('hello server'.encode())

data=client.recv(1024).decode()
print('server says:',data)

client.close()
```

Output:

```
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>cd AppData\Local\Programs\Python\Python313

C:\Users\User\AppData\Local\Programs\Python\Python313>python client.py
server says: hello from server!
```

B) Aim: To write a Python program that generates a properly formatted email message using the built-in email module, including headers such as To, From, Subject, Date, and Message-ID, and outputs the complete email in raw MIME format.

Input:

```
import email.message,email.policy,email.utils,sys

text="welcmoe to advanced python"
```

```
message=email.message.EmailMessage(email.policy.SMTP)
message['To']='you@yourdomain.com'
message['From']='Learn' message['Subject']='A mail to you'
message['Date']=email.utils.formatdate(localtime=True)
message['Messge-ID']=email.utils.make_msgid()
message.set_content(text)
sys.stdout.buffer.write(message.as_bytes())
```

Output:

```
C:\Users\User\AppData\Local\Programs\Python\Python313>python onepython.py
To: you@yourdomain.com
From: Learn
Subject: A mail to you
Date: Thu, 27 Nov 2025 18:30:52 +0530
Messge-ID: <176424845273.5272.5655836729987134269@DESKTOP-FJO594A>
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: 7bit
MIME-Version: 1.0

welcmoe to advanced python
```

C) Aim: Perform a program geocoding using the Nominatim service through the geopy library in Python. It takes the name of a place as input, searches for its exact geographic location, and displays the complete formatted address along with its latitude and longitude coordinates.

Input:

```
from geopy.geocoders import Nominatim
geolocator=Nominatim(user_agent='my_python_app_123(yourmail@gmail.com)')
location=geolocator.geocode("jogeshwari") print(location.address)
print(location.latitude,location.longitude)
```

Output:

```
>
== RESTART: C:\Users\User\AppData\Local\Programs\Python\Python313\onepython.py =
Bhayandar, Railway FOB, Bhayander East, Mira-Bhayander, Thane Taluka, Thane, Maharashtra, 401105, India
19.3101847 72.8526988
>
```

Practical No: 7

Aim: To demonstrate the use of multiple Tkinter widgets and dialogs in a single GUI application.

Input:

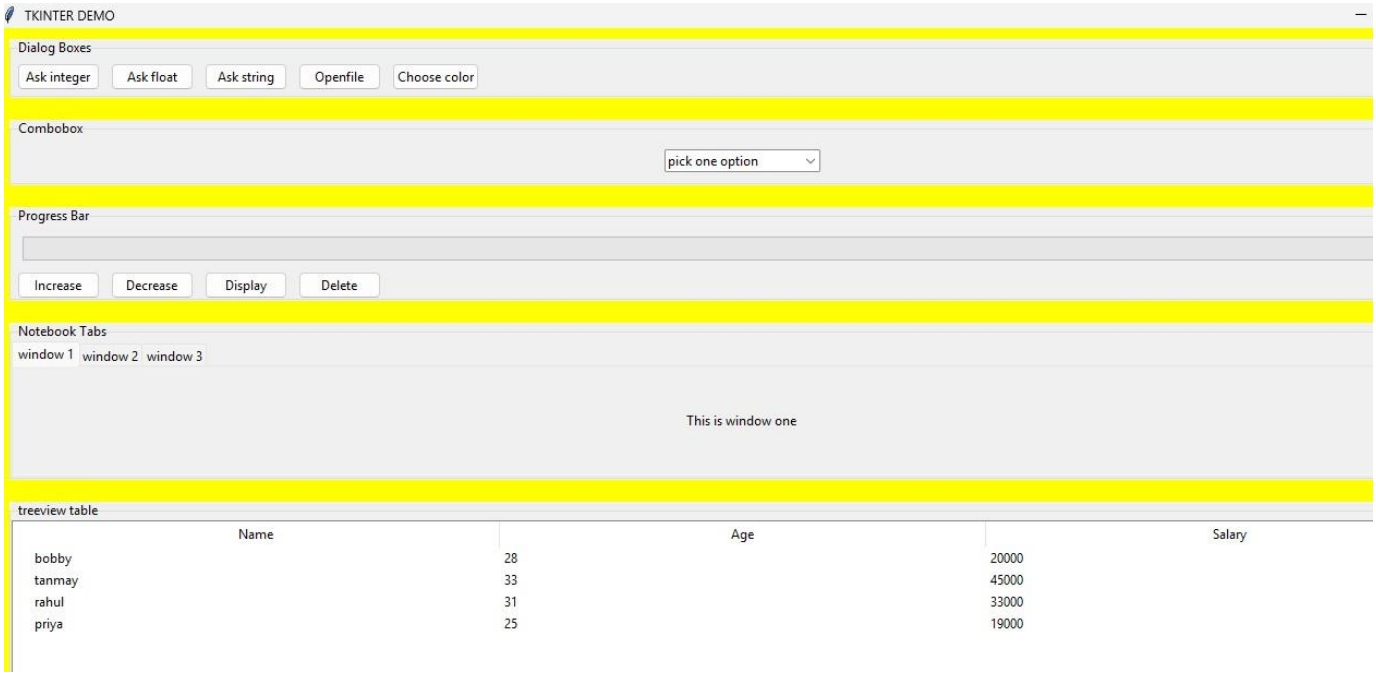
```python
import tkinter as tk
from tkinter import *
from tkinter import ttk
from tkinter.simpledialog import askinteger,askfloat,askstring
from tkinter.filedialog import askopenfile
from tkinter.colorchooser import askcolor

root=tk.Tk()
root.title("TKINTER DEMO")
root.geometry("600x500")
root.config(bg='yellow')
#-----------
#FRAME1
#-----------

dialog_frame=ttk.LabelFrame(root,text="Dialog Boxes")
dialog_frame.pack(padx=10,pady=10,fill="x")
def show_integer():
    num=askinteger("integer input","enter an integer")
    print("integer:",num)

def show_float():
    numfl=askfloat("Float input","enter an float")
    print("Float:",numfl)

def show_string():
    name=askstring("Name input","enter an name")
    print("string:",name)

def open_file():
    f=askopenfile()
    print("File:",f)

def choose_color():
    c=askcolor()
    print("Color:",c)
ttk.Button(dialog_frame,text="Ask integer",command=show_integer).pack(side=LEFT,padx=5,pady=5)
ttk.Button(dialog_frame,text="Ask float",command=show_float).pack(side=LEFT,padx=5,pady=5)
ttk.Button(dialog_frame,text="Ask string",command=show_string).pack(side=LEFT,padx=5,pady=5)
ttk.Button(dialog_frame,text="Openfile",command=open_file).pack(side=LEFT,padx=5,pady=5)
ttk.Button(dialog_frame,text="Choose color",command=choose_color).pack(side=LEFT,padx=5,pady=5)
```

```
#-----------
#FRAME2 combobox
#-----------
combo_frame=ttk.LabelFrame(root,text="Combobox")
combo_frame.pack(padx=10,pady=10,fill="x")
langs=["C","C++","PYTHON","JAVA","MySQL"]
combo=ttk.Combobox(combo_frame,values=langs)
combo.set("pick one option")
combo.pack(padx=10,pady=10)
#-----------
#FRAME3 progressbar
#-----------
progress_frame=ttk.LabelFrame(root,text="Progress Bar")
progress_frame.pack(padx=10,pady=10,fill="x")

def inc():
    progress.step(10)
def dec():
    progress.step(-10)
def delt():
    progress.step(-5)
def disp():
    print("Progress value:",progress["value"])

progress=ttk.Progressbar(progress_frame,mode="determinate")
progress.pack(padx=10,pady=10,fill="x")
ttk.Button(progress_frame,text="Increase",command=inc).pack(side=LEFT,padx=5)
ttk.Button(progress_frame,text="Decrease",command=dec).pack(side=LEFT,padx=5)
ttk.Button(progress_frame,text="Display",command=disp).pack(side=LEFT,padx=5)
ttk.Button(progress_frame,text="Delete",command=delt).pack(side=LEFT,padx=5)
```

```
#-----------
#FRAME4 notebook
#-----------
notebook_frame=ttk.LabelFrame(root,text="Notebook Tabs")
notebook_frame.pack(padx=10,pady=10,fill="both",expand=True)

nb=ttk.Notebook(notebook_frame)
nb.pack(fill="both",expand=True)

#tab 1
tab1=ttk.Frame(nb)
ttk.Label(tab1,text="This is window one").pack(pady=40)
nb.add(tab1,text="window 1")

#tab 2
tab2=ttk.Frame(nb)
ttk.Label(tab2,text="This is window two").pack(pady=40)
nb.add(tab2,text="window 2")

#tab 3
tab3=ttk.Frame(nb)
ttk.Label(tab3,text="This is window three").pack(pady=40)
nb.add(tab3,text="window 3")
```

```python
#-----------
#FRAME5 treeview
#-----------

tree_frame=ttk.LabelFrame(root,text="treeview table")
tree_frame.pack(padx=10,pady=10,fill="both",expand=True)

data=[
    ["bobby",28,20000],
    ["tanmay",33,45000],
    ["rahul",31,33000],
    ["priya",25,19000],
    ]

columns=("age","salary")
tree=ttk.Treeview(tree_frame,columns=columns)
tree.pack(fill="both",expand=True)

tree.heading('#0',text="Name")
tree.heading('age',text="Age")
tree.heading('salary',text="Salary")

#insert rows
for i, row in enumerate(data):
    tree.insert('',"end",iid=i,text=row[0],values=row[1:])

#-----------
#FRAME6 separtor & sizegrip
#-----------

extra_frame=ttk.LabelFrame(root,text="Separator & sizegrip")
extra_frame.pack(padx=10,pady=10,fill='x')

ttk.Label(extra_frame,text="hello world").pack(pady=5)

separator=ttk.Separator(extra_frame,orient=HORIZONTAL)
separator.pack(fill='x',pady=5)

ttk.Label(extra_frame,text="Welcome").pack(pady=5)
size_grip=ttk.Sizegrip(root)
size_grip.pack(anchor=SE)
root.mainloop()
```

Output:



Practical No: 8

Aim: Write a program to show error handling in python.

1. **Try-except**

Input:

```
import sys
while True:
    try:
        n1=int(input("enter number 1:"))
        n2=int(input("enter number 2:"))
        print("the division is:",n1/n2)
    except:
        print("there is an error",sys.exc_info()[0])
```

Output:

```
enter number 1:10
enter number 2:5
the division is: 2.0
enter number 1:4
enter number 2:t
there is an error <class 'ValueError'>
enter number 1:
```

2. **Error as**

argument Input:

```
def Temp_Convert(a):
    try:
        return int(a)
    except ValueError as Arg:
        print("The argument does not contain number\n",Arg)

Temp_Convert("hey")
```

Output:

```
== RESTART: C:/Users/User/AppData/Local/Programs/Python/1
The argument does not contain number
 invalid literal for int() with base 10: 'hey'
```

3. **Custom error**

Input:

```
class NotEqualError(Exception):
    pass
n1=10
n2=12
try:
    if n1!=n2:
        raise NotEqualError
    else:
        print("Number are Equal")
except NotEqualError:
    print("Number does not Equal")
```

Output:

```
== RESTART: C:/Users/User/AppData
Number does not Equal
```

4. **Finally** Input:

```
def Divide(x,y):
    try:
        result=x/y
    except ZeroDivisionError:
        print("--Division--")
    else:
            print("Result is:",result)
    finally:
        print("Final clause is executing")

Divide(21,3)
```

Output:

```
== RESTART: C:/Users/User/AppData/Lo
Result is: 7.0
Final clause is executing
```

Practical No: 9

Aim: Write a program to show different Data Science tools.

1) NumPy:

Input:

```
import numpy as np
x=np.array([1,2,3,4,5,6,7,8,9,10])
print("Array X:",x)
y=np.array([[1,2,3],[4,5,6],[7,8,9]])
print("Array Y:",y)
#array properties
print("Shape:",y.shape)
print("Size:",y.size)
print("Dimension:",y.ndim)

n=np.arange(1,11,2)
print("Array by range:",n)
```

Output:

```
== RESTART: C:\Users\User\AppData\Local\Programs\Py
Array X: [ 1  2  3  4  5  6  7  8  9 10]
Array Y: [[1 2 3]
 [4 5 6]
 [7 8 9]]
Shape: (3, 3)
Size: 9
Dimension: 2
Array by range: [1 3 5 7 9]
```

2) Pandas:

Input:

```
import pandas as pd
df=pd.read_csv(r'mydummy.csv')
print(df)
```

Output:

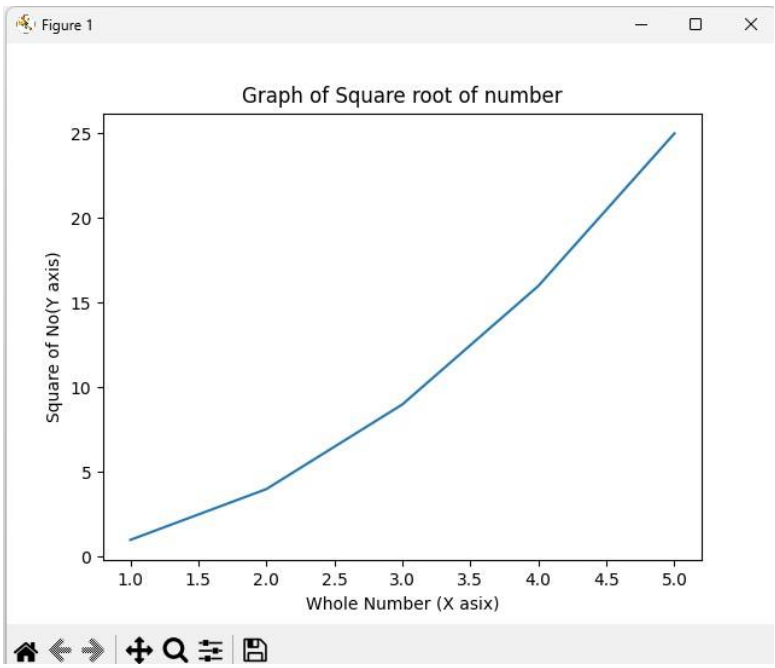
```
== RESTART: C:\Users\User\AppData\Loca
   id   name     age   marks
0   1   Jitesh    20    85
1   2   Neha      21    90
2   3   Drishti   19    78
3   4   Raza      22    88
```

3) Matplot:

Input:

```
import matplotlib.pyplot as plt

x=[1,2,3,4,5]
y=[1,4,9,16,25]

plt.plot(x,y)
plt.xlabel("Whole Number (X asix)")
plt.ylabel("Square of No(Y axis)")
plt.title("Graph of Square root of number")
plt.show()
```

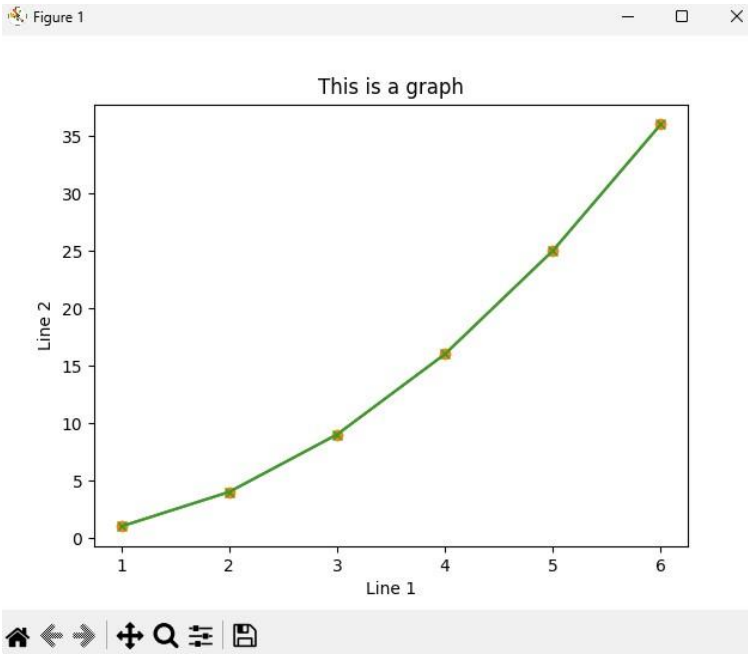
Output:



Input: (with marker)

```
import matplotlib.pyplot as plt
import numpy as np

x_coordinates=np.array([1,2,3,4,5,6])
y_coordinates=np.array([1,4,9,16,25,36])
plt.title("This is a graph")
plt.plot(x_coordinates,y_coordinates)
plt.plot(x_coordinates,y_coordinates,marker='o')
plt.plot(x_coordinates,y_coordinates,marker='x')
plt.xlabel("Line 1")
plt.ylabel("Line 2")
plt.show()
```

Output:



4) SciPy:

a. Double Integral Input:

```
from scipy import integrate
a=lambda y,x: x*y**2
b=lambda x:1
c=lambda x:-1
print(integrate.dblquad(a,0,2,b,c))
```

Output:

```
== RESTART: C:\Users\User\AppData\Local\Programs\Python\P
(-1.3333333333333335, 1.4802973661668755e-14)
```

b. Trimmed Standard

Deviation Input:

```
from scipy import stats
import numpy as np

x=np.arange(20)
print("Trimmed standard deviation:",stats.tstd(x))
print("\n Trimmed standard deviation by setting limit:",stats.tstd(x,(2,10)))
```

Output:

```
== RESTART: C:\Users\User\AppData\Local\Programs\Python\Python313\a
Trimmed standard deviation: 5.916079783099616

 Trimmed standard deviation by setting limit: 2.7386127875258306
```

Practical No: 10

Aim: Write a program to implement various database action in mysql using python.

Input:

1st file db_config

```
import pymysql

def get_connection():
    return pymysql.connect(
        host="localhost",
        user="root",
        password="root",
        database="college",
        port=3306
        )
```

2nd file crud

```python
from db_config import get_connection

def create_student():
    conn=get_connection()
    cur=conn.cursor()

    sid=int(input("Enter ID:"))
    name=input("Enter Name:")
    marks=int(input("Enter marks:"))

    cur.execute(
        "INSERT INTO student VALUES(%s,%s,%s)",(sid,name,marks)
        )

    conn.commit()
    conn.close()
    print("Record Inserted")

def read_student():
    conn=get_connection()
    cur=conn.cursor()

    cur.execute("SELECT*FROM student")
    rows=cur.fetchall()

    if rows:
        for rows in rows:
            print(row)

        else:
            print("No record found")

            conn.close()
```

```python
def update_student():
    conn=get_connection()
    cur=conn.cursor()

    sid=int(input("Enter ID to update:"))
    marks=int(input("Enter marks is update:"))

    cur.execute(
        "UPDATE student SET name=%s,marks=%s WHERE id=%s",(name,marks,sid)
        )
    conn.commit()
    conn.close()
    print("Record Updated")


def delete_student():
    conn=get_connection()
    cur=conn.cursor()

    sid=int(input("Enter ID to delete:"))

    cur.execute(
        "DELETE FROM student WHERE id=%s",(sid)
        )
    conn.commit()
    conn.close()
    print("Record Deleted")


#-------menu--------

while True:
    print("\n---STUDENT DATABASE MENU---")
    print("1.Insert Stduent")
    print("2.View Stduent")
    print("3.Update Stduent Mark")
    print("4.Delete Stduent")
    print("5.Exit")

    choice=input("Enter choice:")

    if choice=="1":
        create_student()

    elif choice=="2":
        read_students()

    elif choice=="3":
        update_student()

    elif choice=="4":
        delete_student()

    elif choice=="5":
        print("Exiting program")
        break
    else:
        print("Invaild Choice.")
```

```
==== RESTART: C:\Users\User\AppData\Local\

---STUDENT DATABASE MENU---
1.Insert Stduent
2.View Stduent
3.Update Stduent Mark
4.Delete Stduent
5.Exit
Enter choice:1
Enter ID:102
Enter Name:Rahul
Enter marks:69
Record Inserted
```

25

Result Grid | Filter Rows:

sid	name	marks
101	Nobita	70
102	Rahul	69

<--

Input: